What is Supabase? (The "Firebase Alternative")

The Concept

Supabase is an **Open Source** Backend-as-a-Service (BaaS).

The Core

Unlike Firebase (which uses NoSQL), Supabase is built on top of **PostgreSQL**, a powerful relational database.

Why use it?

- It gives you instant APIs (REST & Realtime).
- It handles Authentication, Storage, and Edge Functions.
- Great for developers who want the power of SQL with the ease of a cloud service.

Setting Up the Project

01

The Dashboard

Go to **supabase.com**, create a project, and set your database password.

02

The Credentials

You need two things from your Project Settings:

1. **Project URL:** The endpoint where your database lives.
2. **Anon Key:** The public API key used for client-side requests.

03

Installation

In your web project folder, install the library using npm:

```
npm install @supabase/supabase-js
```

Initializing the Client

To make your JavaScript app "talk" to your database,you must initialize the Supabase client.

The Code:

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = 'YOUR_SUPABASE_URL'
const supabaseKey = 'YOUR_SUPABASE_ANON_KEY'

const supabase = createClient(supabaseUrl, supabaseKey)
```

Note: Always use environment variables (.env) for your keys in production projects to keep them secure!

Reading and Writing Data (CRUD)

Since it's based on PostgreSQL, the syntax feels very logical and "human-readable."

Inserting Data:

```
const { data, error } = await supabase
.from('menu_items') // Your table name
.insert([{ name: 'Classic Burger', price: 10 }])
```

Fetching Data:

```
const { data: items, error } = await supabase
.from('menu_items')
.select('*')
.eq('available', true) // Filter: only show available items
```

Managing Authentication

Supabase makes login/signup incredibly simple. It handles everything from email confirmation to social logins (Google, GitHub).

Sign Up:

```
const { user, error } = await supabase.auth.signUp({
email: 'example@email.com',
password: 'password123',
})
```

Login:

Uses signInWithPassword. Once logged in, Supabase manages the "Session" automatically, so the user stays logged in even if they refresh the page.

Realtime Updates

One of the coolest features for webapps (like a restaurant order tracker) is the Realtime engine.

Listening for changes:

```
const mySubscription = supabase
.channel('public:menu_items')
.on('postgres_changes', { event: 'UPDATE',
schema: 'public',
 table: 'menu_items'
 }, payload => {
 console.log('Menu updated!', payload)
 })  .subscribe()
```

The Result:

Your website can update instantly without the user ever clicking "Refresh."